

Trekking Management Application

1. Student Details

Name: Neeraj Kushwaha

Roll Number: 25f2009320

Email: 25f2009320@ds.study.iitm.ac.in

About Me: I am a student of the BS Degree in Data Science and Applications at IIT Madras. I enjoy learning web development and building practical applications using Python, Flask, SQLAlchemy, and Bootstrap. This project helped me improve my understanding of backend development, database design, and role-based web applications.

2. AI / LLM Usage

Component	Purpose	Approx. AI Usage
Bootstrap UI & Layout	Responsive UI design and page styling	5% - 7%
HTML/Jinja Templates	Template formatting and layout improvements	8% - 10%
Backend Logic	Small route logic suggestions and code improvements	5% - 7%
Bug Fixing	Understanding errors and debugging support	3% - 5%
Documntation	Report writing and documentation assistance	1% - 3%
Feature Implementation Support	Understanding implementation approach for selected features (search, profile editing, activation/deactivation, booking improvements)	5% - 7%

Declaration

AI (ChatGPT GPT- 5.5) was used mainly to understand concepts, improve Bootstrap layouts, debug errors, and get suggestions for implementing some project features. The application architecture, database schema, route integration, testing, debugging, and final implementation were completed manually.

3. Project Details

Project Title: Trekking Management Application

Problem Statement: To develop a web-based Trekking Management Application that allows Admin, Trek Staff, and Users to manage trekking activities efficiently through role-based access.

Approach: The application is developed using Flask with SQLite as the database. It provides separate dashboards for Admin, Staff, and Users. Users can book treks, Staff can manage assigned treks, and Admin can manage treks, users, staff, and bookings through a centralized dashboard

4. Technologies Used

Technology	Purpose
Python	Used as the main programming language for backend development.
Flask	Used to build the web application, handle routing, and process user requests.
SQLAlchemy	ORM to perform database operations without writing raw SQL queries
SQLite	Lightweight local database for storing user data
Jinja2	Used to render dynamic HTML pages with data from the backend
HTML5	Used to create the structure of web pages

5. Database Schema / ER Diagram

The application uses **SQLite** as the database with **Flask-SQLAlchemy** for database management.

Tables

User — Stores information about Admin, Trek Staff, and Users (id, name, email, password, role, approved, active).

Trek — Stores trek details (id, name, location, difficulty, duration, total_slots, available_slots, status, staff_id).

Booking — Stores booking information (id, user_id, trek_id, booking_status, payment_status, booking_date).

Announcement — Stores announcements displayed on the dashboards (id, message).

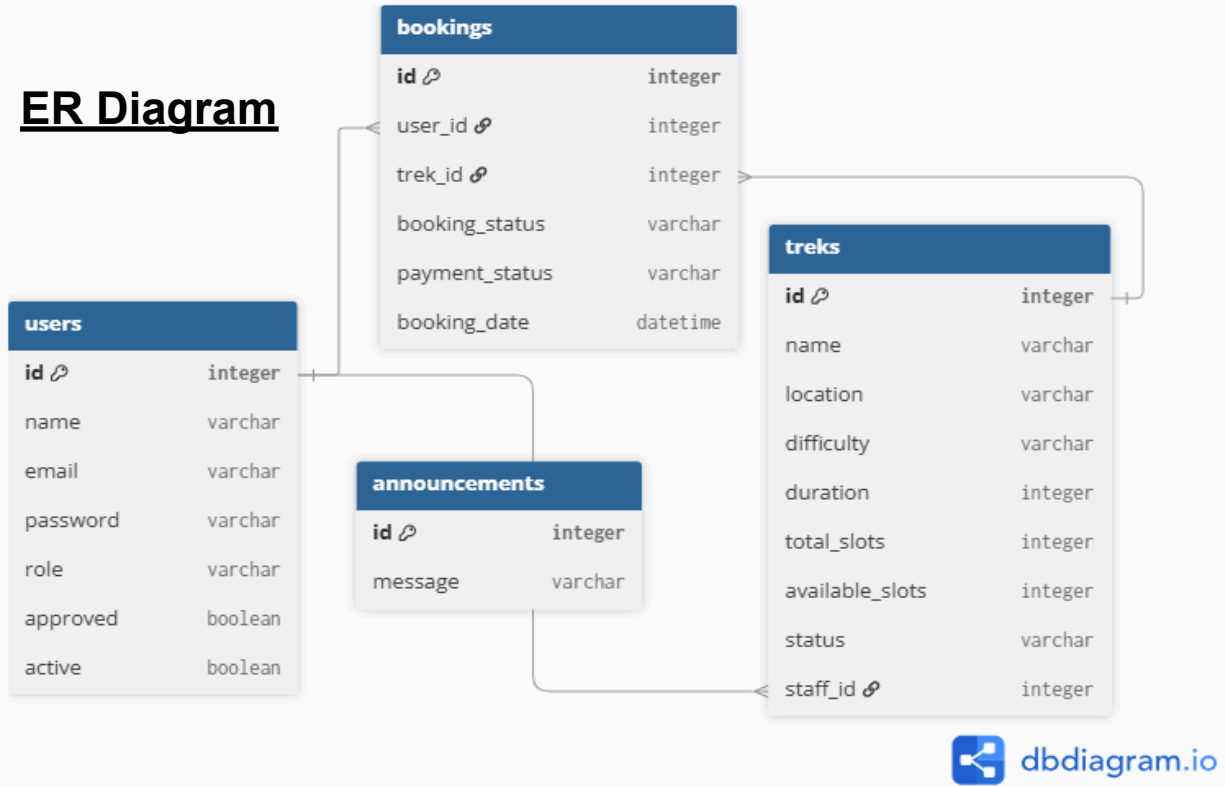
Relationships

- **One-to-Many** → **User** → **Trek**
 - One staff member can manage multiple treks.
- **One-to-Many** → **User** → **Booking**
 - One user can make multiple bookings.
- **One-to-Many** → **Trek** → **Booking**
 - One trek can have multiple bookings.
- **Many-to-One** → **Booking** → **User**
 - Each booking belongs to one user.
- **Many-to-One** → **Booking** → **Trek**
 - Each booking belongs to one trek.

Database Design

The database is designed to maintain proper relationships between users, treks, and bookings while reducing data redundancy. Foreign keys are used to ensure data consistency and integrity.

ER Diagram



6. API Design

The application uses Flask routes to handle user authentication, trek management, booking management, and profile management. A separate **api.yaml** file containing the API definitions is included in the project submission.

API Resources:

- Login
- Registration
- Add Trek
- Edit Trek
- Delete Trek
- Book Trek
- Cancel Booking
- Edit Profile
- View Users
- View Staff

7. Architecture and Features

Architecture

- `app.py` – Application routes and business logic
- `models.py` – Database models and relationships
- `templates/` – Jinja2 HTML templates
- `instance/` – SQLite database
- `config.py` – Application configuration

Features

- Admin, Staff, and User Authentication
- Role-Based Access Control
- Trek Management
- Trek Booking
- Booking Cancellation
- Staff Approval
- Staff Assignment
- Trek Status Management
- User Profile Editing
- Search and Filter Treks
- etc.

8. Video

<https://drive.google.com/drive/folders/1G14Vb3YeUUnJknPtYUC9DkT2MUck3DaB>